

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Computação Gráfica

Ano Letivo de 2025/2026

Fase 1 - Gerador e Motor 3D

Carlos Araújo
A106845

João Teixeira
A106836

Simão Mendes
A106928

Vítor Senra
A106921

February 26, 2026

CG

Índice

1. Introdução	1
2. Gerador (<i>Generator</i>) - Modelação e Matemática	2
2.1. Estrutura Geral do Sistema	2
2.2. Primitiva: <i>Plane</i>	2
2.2.1. Definição	2
2.2.2. Geração de Vértices	2
2.3. Primitiva <i>Box</i>	3
2.3.1. Definição	3
2.3.2. Geração de Vértices	3
2.4. Primitiva: Esfera	3
2.4.1. Definição e Modelação Matemática	3
2.4.2. Coordenadas Esféricas	3
2.5. Primitiva: Cone	4
2.5.1. Definição	4
2.5.2. Superfície Lateral	4
2.6. Formato Customizado .3d	4
3. Motor 3D (<i>Engine</i>) e Estruturas de Dados	5
3.1. Criação da Janela e Contexto <i>OpenGL</i>	5
3.2. Arquitetura e <i>Parsing XML</i>	5
3.3. A Câmara e o Espaço de Visualização	6
3.4. Estruturas de Dados e Gestão de Memória	6
3.5. Comportamento do Motor e Ciclo de Renderização	7
3.5.1. Limitações Atuais e Trabalho Futuro (Otimização de <i>Cache</i>)	7
4. Resultados e Validação	9
4.1. Resultados e Validação	9
4.1.1. Teste 1.1: Cone (<i>cone</i>)	9
4.1.2. Teste 1.2: Posição da Câmara	10
4.1.3. Teste 1.3: Esfera (<i>sphere</i>)	10
4.1.4. Teste 1.4: Caixa (<i>box</i>)	11
4.1.5. Teste 1.5: Esfera e plano (<i>sphere & plane</i>)	11
4.1.6. Testes com Modelos Externos	11
4.1.6.1. Teste de Stress: Catedral de <i>Sibenik</i>	12
4.1.6.2. Teste de Superfícies Orgânicas: Modelo <i>Blub</i>	12
5. Conclusão	14
5.1. Desafios Encontrados	14
6. Referências Bibliográficas	15

7. Anexos	16
7.1. Anexo A - Estrutura Interna do Ficheiro .3d (Exemplo)	16
7.2. Anexo B - <i>Script</i> de Conversão (.obj para .3d)	17

1. Introdução

A primeira fase deste projeto consistiu no desenvolvimento de um **motor gráfico** construído de raiz em linguagem **C++**. O objetivo principal foi a criação de um sistema capaz de **gerar, carregar e visualizar** modelos tridimensionais num ambiente virtual configurável. Para garantir a eficiência e a organização do código, a arquitetura foi dividida em dois componentes principais:

- **Generator:** Este módulo funciona como uma **ferramenta independente** dedicada à modelação matemática. A sua função é **transformar superfícies contínuas em malhas de triângulos**, que são a unidade básica de desenho das placas gráficas. O programa calcula os vértices de figuras como o Plano, a Caixa, a Esfera e o Cone, e guarda esses dados num ficheiro de formato próprio (.3d).
- **Engine:** Este componente é o motor de visualização que **consome os dados gerados**. O motor utiliza a biblioteca **tinyxml2** para ler um ficheiro de configuração XML, onde estão definidos os modelos a carregar e as propriedades da câmara, como a sua posição e o campo de visão. Com base nestes dados, a *Engine* utiliza o OpenGL para desenhar as cenas no ecrã através de uma representação em *wireframe*.

Esta divisão de tarefas permite que os cálculos geométricos mais pesados ocorram apenas uma vez, durante a fase de geração. Desta forma, o motor de renderização liberta-se de processamento redundante e foca-se exclusivamente na **leitura rápida dos ficheiros para a memória RAM** e na sua **exibição fluida** para o utilizador.

2. Gerador (*Generator*) - Modelação e Matemática

O *Generator* atua como uma ferramenta autónoma cujo propósito é discretizar as superfícies contínuas de objetos em múltiplos triângulos (a primitiva de desenho nativa da placa gráfica).

2.1. Estrutura Geral do Sistema

O sistema de geração implementa um padrão de design polimórfico onde cada primitiva herda de uma classe abstrata *Figure*. O fluxo de execução é:

1. **Parsing dos argumentos** de linha de comando para identificar o tipo de primitiva.
2. **Instanciação da classe** correspondente com os parâmetros específicos.
3. **Chamada do método `generate()`** para criar a malha de vértices.
4. **Escrita dos vértices num ficheiro .3d** através do método `writeToFile()`.

2.2. Primitiva: *Plane*

```
./generator plane <length> <divisions> <file>
```

2.2.1. Definição

A primitiva *plane* é designada por um quadrado de lado *length* no plano XZ, centrado na origem e com subdivisões no eixos X e Z. O número de subdivisões por eixo é dado pelo argumento *divisions*.

2.2.2. Geração de Vértices

A estratégia utiliza uma grelha bidimensional onde cada célula é dividida em dois triângulos com orientação *counter-clockwise*.

$$\Delta = \frac{\text{length}}{\text{divisions}}$$

$$\text{inicio} = -\frac{\text{length}}{2}$$

Pseudo-código:

```
1 Para i de 0 até divisions - 1:
2     Para j de 0 até divisions - 1:
3         x1 = inicio + i * Delta; z1 = inicio + j * Delta
```

text

```

4          x2 = inicio + (i+1) * Delta; z2 = inicio + (j+1) * Delta
5
6          Triangulo1: (x1, 0, z1), (x1, 0, z2), (x2, 0, z2)
7          Triangulo2: (x1, 0, z1), (x2, 0, z2), (x2, 0, z1)

```

2.3. Primitiva *Box*

`./generator box <length> <grid> <file>`

2.3.1. Definição

Um cubo centrado na origem, estendendo-se de $-\frac{\text{length}}{2}$ a $\frac{\text{length}}{2}$ em todos os eixos. Cada face possui uma subdivisão de *grid times grid*.

2.3.2. Geração de Vértices

A geração da caixa aplica a lógica do *Plane* em seis orientações distintas. Para cada face, um eixo permanece fixo (ex: $y = \frac{\text{length}}{2}$ para a face superior) enquanto os outros dois variam.

2.4. Primitiva: Esfera

`generator sphere <radius> <slices> <stacks> <file>`

2.4.1. Definição e Modelação Matemática

A esfera é gerada através de coordenadas esféricas, com uma subdivisão em *slices* (longitude) e *stacks* (latitude). A implementação segue o padrão matemático clássico, onde a geração ocorre do Polo Norte para o Polo Sul. Para otimizar a geometria, as extremidades (polos) são seladas com triângulos únicos, enquanto o corpo da esfera utiliza pares de triângulos para formar cada face da malha.

2.4.2. Coordenadas Esféricas

As coordenadas de cada vértice são calculadas com base no raio r , no ângulo azimutal α e no ângulo polar β :

$$x = r \cdot \sin(\beta) \cdot \sin(\alpha)$$

$$y = r \cdot \cos(\beta)$$

$$z = r \cdot \sin(\beta) \cdot \cos(\alpha)$$

Onde os intervalos dos ângulos são:

- $\alpha \in [0, 2\pi]$ (correspondente às *slices*)

- $\beta \in [0, \pi]$ (correspondente às *stacks*)

2.5. Primitiva: Cone

`./generator cone <radius> <height> <slices> <stacks> <file>`

2.5.1. Definição

O cone combina uma base circular no plano $y=0$ e uma superfície lateral que converge para o ápice em $(0, height, 0)$.

2.5.2. Superfície Lateral

O raio em cada nível (*stack*) é proporcional à altura:

$$r(s) = \text{radius} \cdot \left(1 - \frac{s}{\text{stacks}}\right)$$

$$y(s) = s \cdot \left(\frac{\text{height}}{\text{stacks}}\right)$$

Pseudo-código (Lateral):

```

1 Para s de 0 até stacks - 1:
2     r_atual = r(s); r_seg = r(s+1)
3     y_atual = y(s); y_seg = y(s+1)
4     Para i de 0 até slices:
5         alfa1 = i * Delta; alfa2 = (i+1) * Delta
6         Gerar faces ligando (r_atual, alfa1) -> (r_seg, alfa1) ->
           (r_seg, alfa2)
```

text

2.6. Formato Customizado .3d

Para permitir a comunicação entre o *Generator* e o *Engine*, criou-se o formato binário/texto .3d. A estrutura do ficheiro organiza-se pela especificação inicial do número total de vértices na primeira linha, seguida de sucessivas linhas contendo as componentes cartesianas (x, y, z) de cada vértice. Esta arquitetura sequencial agiliza as rotinas de I/O em fase de leitura.

3. Motor 3D (*Engine*) e Estruturas de Dados

O *Engine* atua como o *software* consumidor dos dados. A sua arquitetura visa separar de forma limpa a fase de inicialização (onde ocorrem as operações de leitura de disco e *parsing*, que são computacionalmente pesadas) da fase do ciclo de desenho (*render loop* usando *GLUT*), que necessita de correr nativamente a múltiplos *frames* por segundo.

3.1. Criação da Janela e Contexto *OpenGL*

O primeiro passo na execução do motor é a inicialização do contexto visual. Utilizando a biblioteca *GLUT*, o programa extrai as dimensões pretendidas a partir do ficheiro XML e configura o modo de visualização com as seguintes propriedades operacionais:

- **Testes de Profundidade:** O motor ativa o *Depth Test* (`GL_DEPTH_TEST`) para garantir a correta sobreposição espacial dos objetos tridimensionais e evitar artefactos visuais;
- **Otimização Geométrica:** É ativado o *Culling* de faces ocultas (`GL_CULL_FACE`), uma otimização fundamental que descarta a renderização de polígonos não voltados para a câmara, poupando recursos;
- **Transições Fluidas:** O *display mode* é configurado com *Double Buffering* (`GLUT_DOUBLE`) para promover transições de imagem suaves e sem oscilação (*flickering*);
- **Modo de Visualização:** Nesta primeira fase, a renderização foi configurada para operar estritamente em modo *wireframe* (através do comando `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)`), permitindo uma verificação gráfica clara da topologia e da correta subdivisão dos triângulos gerados.

3.2. Arquitetura e *Parsing* XML

A configuração do ambiente 3D é carregada de forma dinâmica através da leitura de um ficheiro XML, utilizando a biblioteca externa **tinyxml2**. O fluxo de execução do motor (função auxiliar `loadXML`) inicia-se extraindo a informação hierárquica a partir da **tag** raiz `<world>`:

- **Janela (`<window>`):** O motor extrai a resolução pretendida (atributos `width` e `height`), aplicando o valor por defeito de 512x512 caso a **tag** seja omitida;
- **Câmara (`<camera>`):** O *parser* navega pelas *sub-tags* `<position>`, `<lookAt>` e `<up>` para armazenar as coordenadas espaciais `x`, `y` e `z`. Simultaneamente, lê a **tag** `<projection>` (`fov`, `near`, `far`) para configurar os recortes da perspetiva;
- **Modelos (`<group>` e `<models>`):** Iterando ciclicamente sobre os nós `<model>`, o sistema extrai o caminho relativo (atributo `file`) de cada ficheiro `.3d` gerado previamente pelo *Generator*.

Abaixo encontra-se um exemplo da estrutura lida pelo motor, correspondente ao ficheiro de configuração `test_1_1.xml`:

```
1  <world>
2    <window width="512" height="512" />
3    <camera>
4      <position x="5" y="-2" z="3" />
5      <lookAt x="0" y="0" z="0" />
6      <up x="0" y="1" z="0" />
7      <projection fov="60" near="1" far="1000" />
8    </camera>
9    <group>
10     <models>
11       <model file="cone_1_2_4_3.3d" />
12     </models>
13   </group>
14 </world>
```

3.3. A Câmara e o Espaço de Visualização

Para abstrair a complexidade matemática da visualização, criou-se a classe `Camera`. Esta estrutura encapsula os vetores de posição, direção (*lookAt*) e orientação vertical (*up*). O cálculo das matrizes decorre de forma sequencial no método `Camera::apply()`, dividindo-se em duas etapas essenciais:

- **Matriz de Projeção (GL_PROJECTION):** Numa primeira fase, o motor calcula dinamicamente o *aspect ratio* da janela e ativa a matriz de projeção, alimentando a função nativa `gluPerspective` com o *FOV* e os planos de recorte geométrico;
- **Matriz de Visualização (GL_MODELVIEW):** De seguida, o sistema comuta o estado da matriz e define o enquadramento do mundo através da função `gluLookAt`.

Esta separação rigorosa de responsabilidades assegura que as transformações do ponto de vista da cena são aplicadas corretamente antes de se iniciar o desenho de qualquer geometria.

3.4. Estruturas de Dados e Gestão de Memória

Para cumprir o requisito de otimização que dita que os modelos tridimensionais do disco apenas podem ser acedidos **uma única vez** no arranque da aplicação, foi desenhada uma arquitetura robusta baseada, maioritariamente, em contentores da *Standard Template Library* (STL).

A lógica de carregamento e gestão da memória opera nos seguintes passos durante a fase de **Setup**:

1. O motor acede a cada ficheiro `.3d` e lê a **primeira linha**, que contém um inteiro com o **número total de vértices** (`numPoints`);
2. Instancia-se um objeto da classe `Model`, que internamente contém um **vetor dinâmico** (`std::vector<Point>`). O tipo `Point` é uma **estrutura simples** (*struct*) que **encapsula as coordenadas** `x`, `y` e `z`;
3. Um ciclo extrai sequencialmente as coordenadas do ficheiro de texto, guardando os pontos na memória RAM. Após a leitura, o **stream** de dados do ficheiro é imediatamente fechado;
4. O modelo devidamente carregado é adicionado a uma **lista global de modelos** (`std::vector<Model> myModels`).

3.5. Comportamento do Motor e Ciclo de Renderização

No ciclo principal de renderização do *OpenGL* (`renderScene`), a placa gráfica **consome exclusivamente a informação já residente na RAM**. O procedimento inicia-se com a limpeza dos *buffers* de cor e profundidade. Após a aplicação das matrizes da câmara, o motor desenha os eixos cartesianos absolutos (com cores distintas para `X`, `Y` e `Z`) para providenciar uma referência espacial para o cenário.

Seguidamente, o programa limita-se a iterar sobre as instâncias armazenadas no contentor `myModels`. Cada instância invoca o seu método `draw()`, submetendo iterativamente os vértices ao *pipeline* gráfico através dos comandos `glBegin(GL_TRIANGLES)` e `glVertex3f`. O ciclo culmina com a troca de *buffers* visuais. Esta separação de arquitetura garante uma **operação fluida**, inteiramente livre de bloqueios de *I/O* (entradas e saídas de disco).

3.5.1. Limitações Atuais e Trabalho Futuro (Otimização de *Cache*)

Apesar de a arquitetura atual garantir isolamento total entre leituras de disco e a rotina de renderização, a estrutura iterativa de **parsing** atual padece de uma redundância de alocação de memória:

- **O Problema:** Caso a cena requeira o desenho de múltiplos modelos idênticos (por exemplo, dezenas de instâncias de uma mesma esfera), o motor acede repetidamente ao disco e aloca a geometria em duplicado na memória RAM, tantas vezes quantas as instâncias pedidas.
- **Trabalho Futuro:** Para a próxima fase do projeto, com a introdução do Grafo de Cena, planeia-se mitigar este consumo linear de recursos através da implementação de um **sistema de Cache de modelos** (com recurso a um dicionário `std::unordered_map`). Desta forma, garantir-se-á que cada ficheiro `.3d` é lido e instanciado apenas na sua primeira ocorrência, sendo a sua zona de memória partilhada pelas restantes entidades do cenário.

A Figura abaixo ilustra o mapeamento conceptual das classes criadas para garantir a persistência dos dados e a sua relação com o estado global do motor:

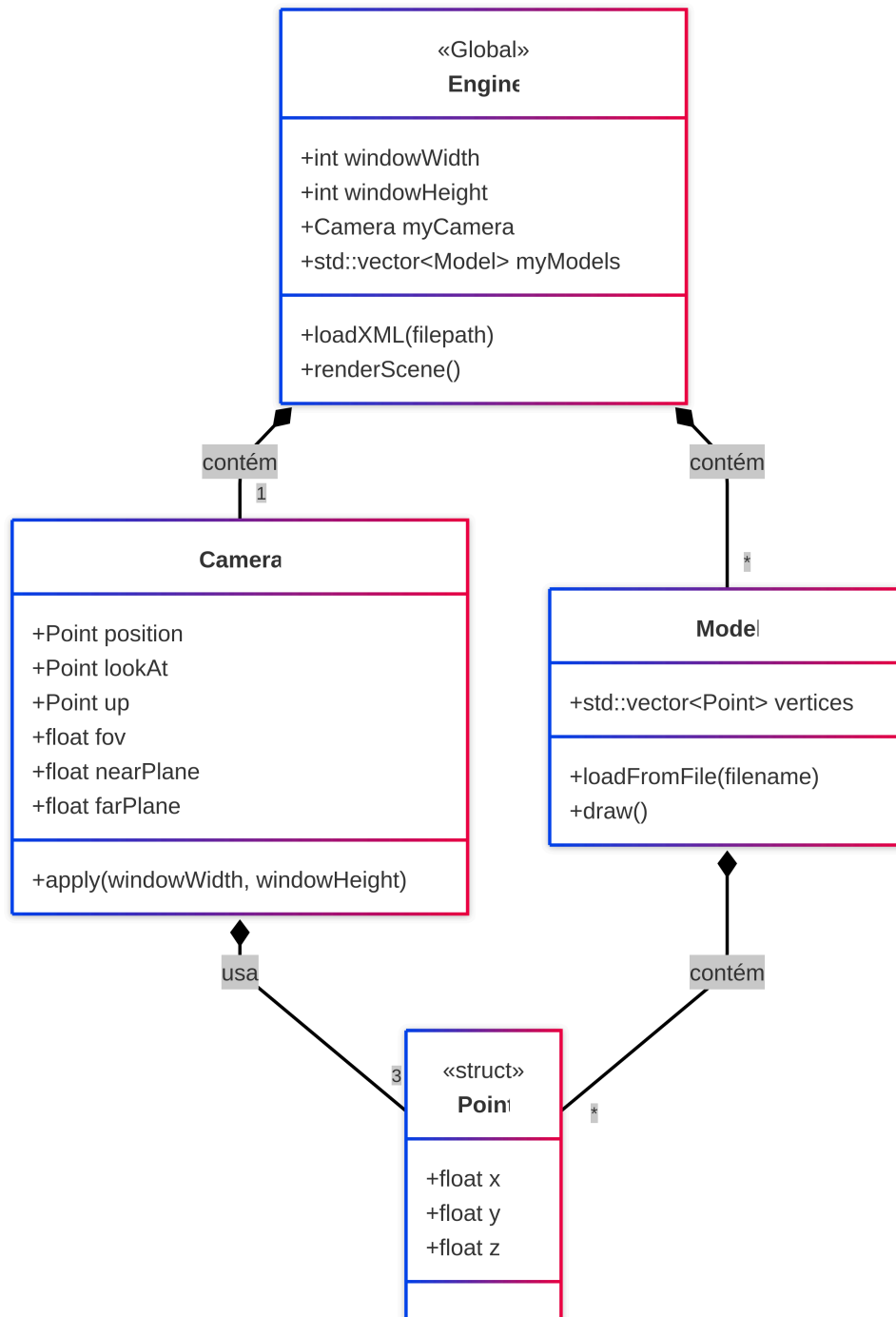


Figura 1: Diagrama de Classes UML ilustrando a gestão de geometria em memória RAM (suprimindo acessos redundantes ao disco).

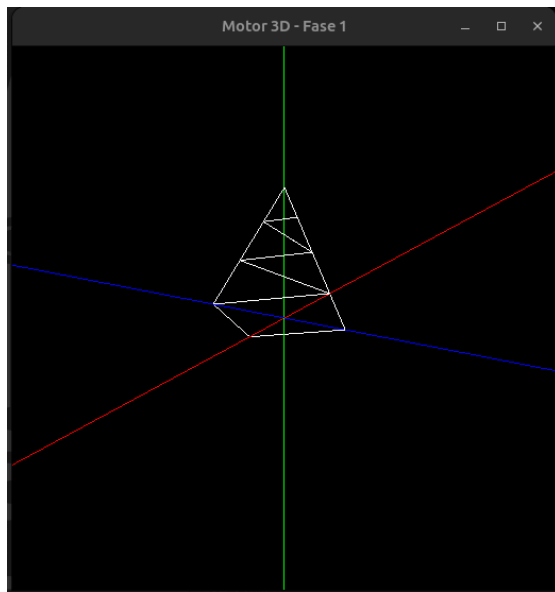
4. Resultados e Validação

Nesta secção, apresenta-se a **validação visual do projeto**. Para cada teste, foi executado o comando correspondente no *Generator*, seguido da visualização do ficheiro resultante na *Engine*. As imagens à esquerda representam o *output* obtido pelo nosso programa, enquanto as imagens à direita são as referências fornecidas pela equipa docente. A correspondência total entre as imagens confirma a correta implementação das primitivas.

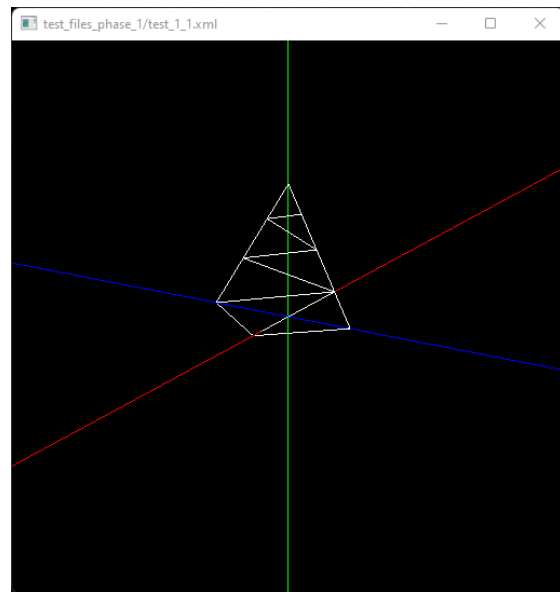
4.1. Resultados e Validação

Nesta secção, comprovamos que o código desenvolvido funciona corretamente segundo os requisitos. Corremos o gerador e o motor para os testes fornecidos e comparámos os resultados visualmente.

4.1.1. Teste 1.1: Cone (*cone*)



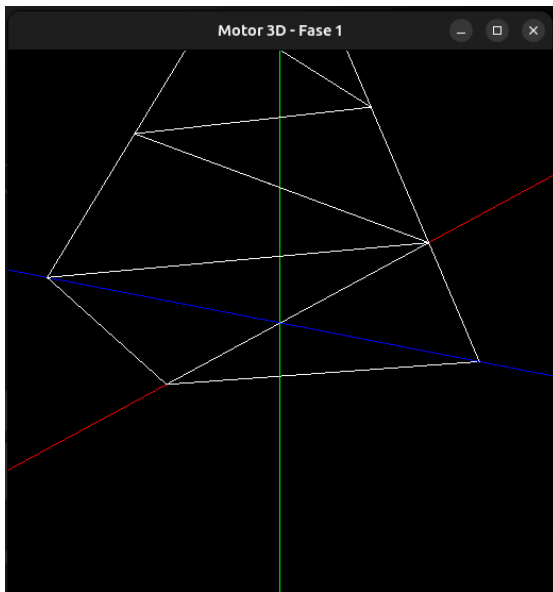
O Nosso Programa



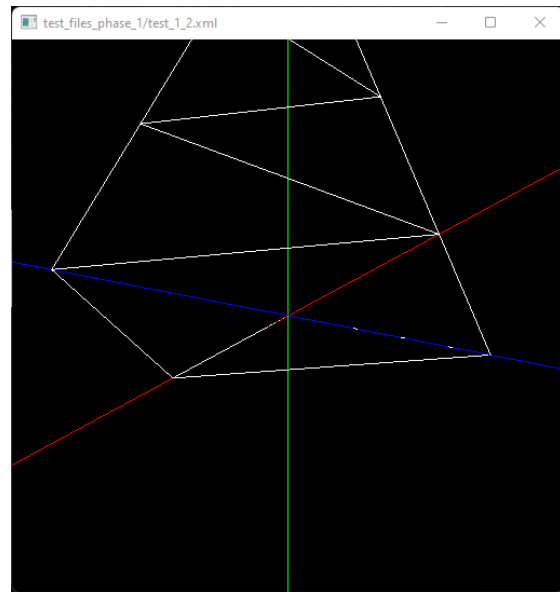
Referência do Professor

Figura 2: Comparação visual para o teste 1_1.

4.1.2. Teste 1.2: Posição da Câmera



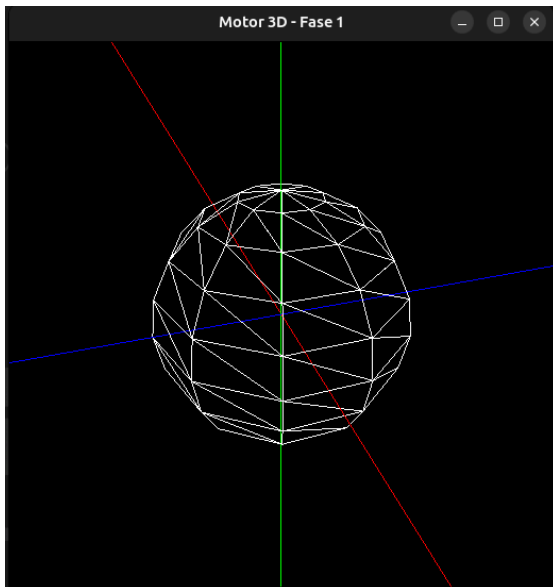
O Nosso Programa.



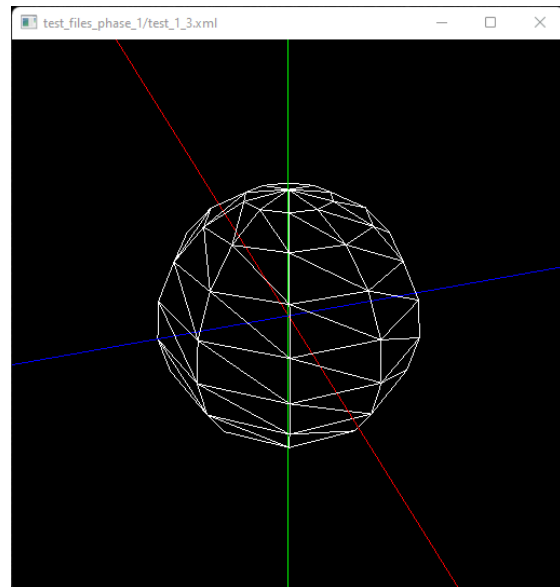
Referência do Professor.

Figura 3: Comparação visual para o teste 1_2.

4.1.3. Teste 1.3: Esfera (*sphere*)



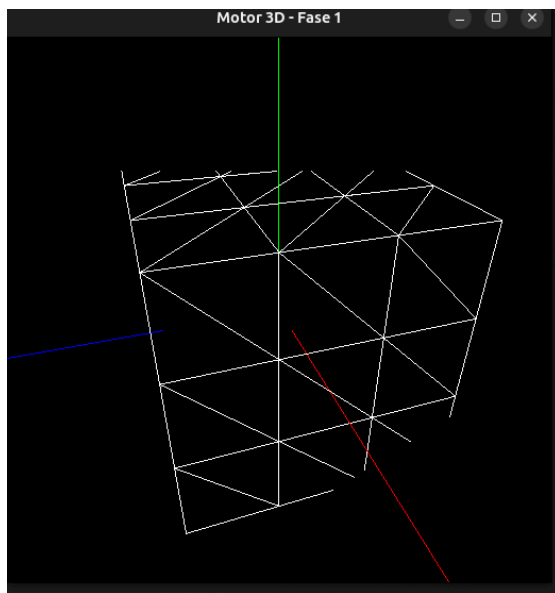
O Nosso Programa.



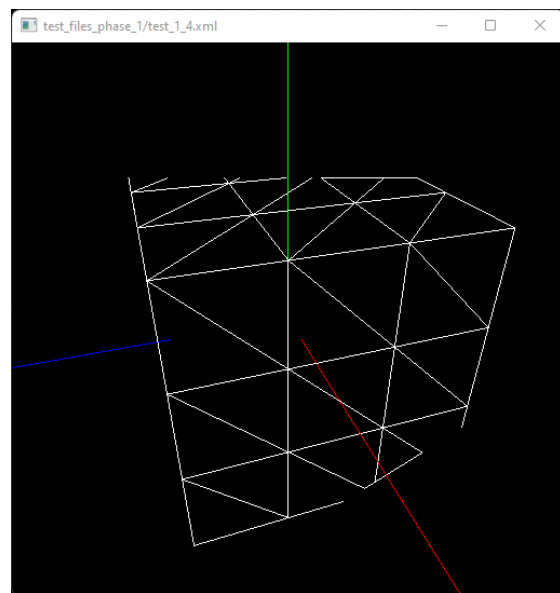
Referência do Professor.

Figura 4: Comparação visual para o teste 1_3.

4.1.4. Teste 1.4: Caixa (*box*)



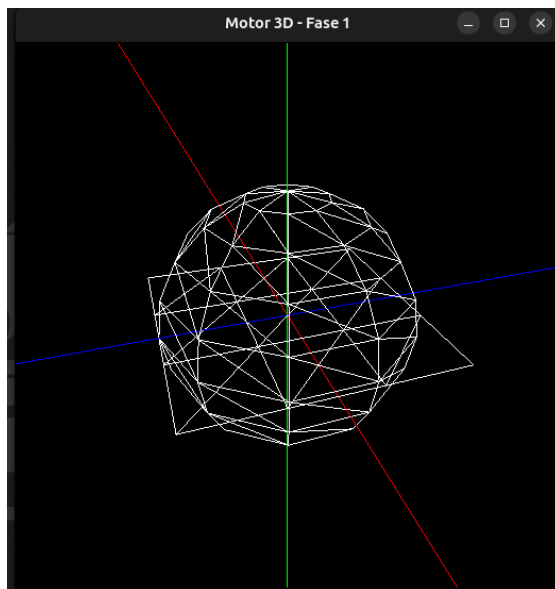
O Nosso Programa.



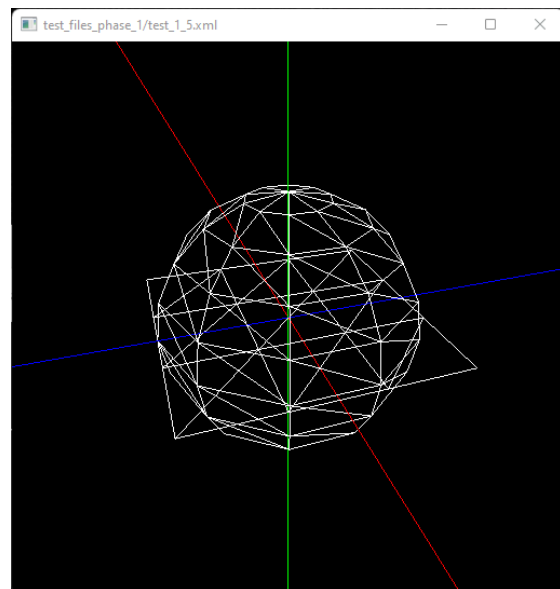
Referência do Professor.

Figura 5: Comparação visual para o teste 1_4.

4.1.5. Teste 1.5: Esfera e plano (*sphere & plane*)



O Nosso Programa.



Referência do Professor.

Figura 6: Comparação visual para o teste 1_5.

4.1.6. Testes com Modelos Externos

Como abordado na secção de limitações, o **motor atual ainda não dispõe de um sistema de *cache* para os modelos geométricos**. De forma a testar a robustez e os limites de renderização da nossa *Engine* perante esta condicionante, desenvolvemos um *script* auxiliar de conversão (detalhado no Anexo B). Este utilitário permitiu-nos trans-

formar modelos complexos obtidos na *internet* (no formato *standard .obj*) para o nosso formato proprietário *.3d*.

Apresentam-se de seguida os resultados da importação destas geometrias externas, que comprovam a capacidade do motor em processar e desenhar um elevado volume de vértices de forma correta e sem anomalias visuais.

4.1.6.1. Teste de Stress: Catedral de *Sibenik*

Para levar a otimização geométrica do nosso motor ao limite, seleccionámos o modelo ***Sibenik Cathedral*** do arquivo de **Computação Gráfica de McGuire**. Este modelo é composto por **225852 vértices**, representando um salto de complexidade brutal face às primitivas geradas anteriormente.

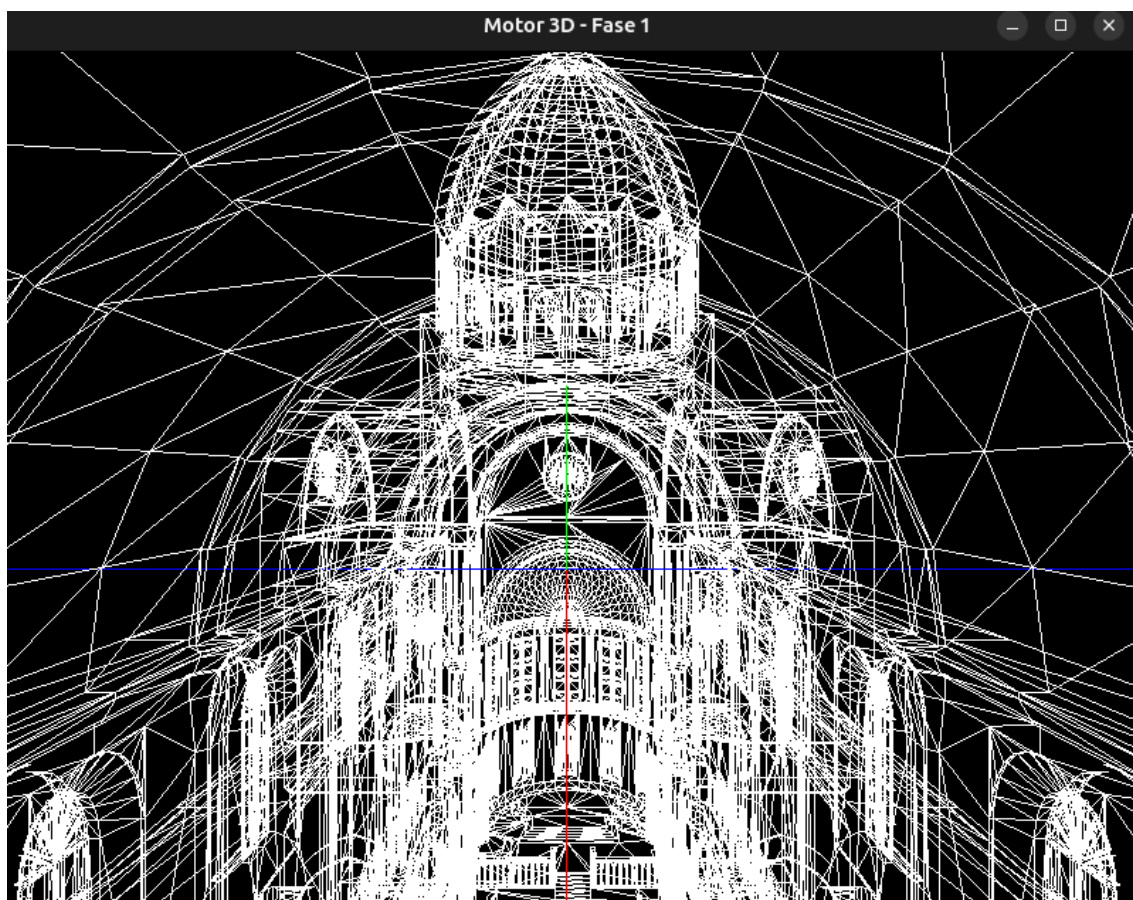


Figura 7: Catedral de *Sibenik*.

4.1.6.2. Teste de Superfícies Orgânicas: Modelo *Blub*

Para diversificar a validação do nosso motor e do *script* de conversão *.obj* para *.3d*, decidimos testar um modelo de natureza orgânica, contrastando com as linhas rígidas e estruturais da geometria anterior, este tem **42624 vértices**. O modelo escolhido foi o ***Blub*** (um peixe frequentemente utilizado na área de Computação Gráfica para testes de subdivisão geométrica e texturização).

Este teste é particularmente relevante para avaliar o comportamento do motor ao renderizar uma elevada densidade de curvas suaves. Como a visualização da Fase 1 opera

estritamente em *wireframe*, a topologia da malha orgânica e a forma como os triângulos descrevem os contornos do modelo ficam perfeitamente visíveis.

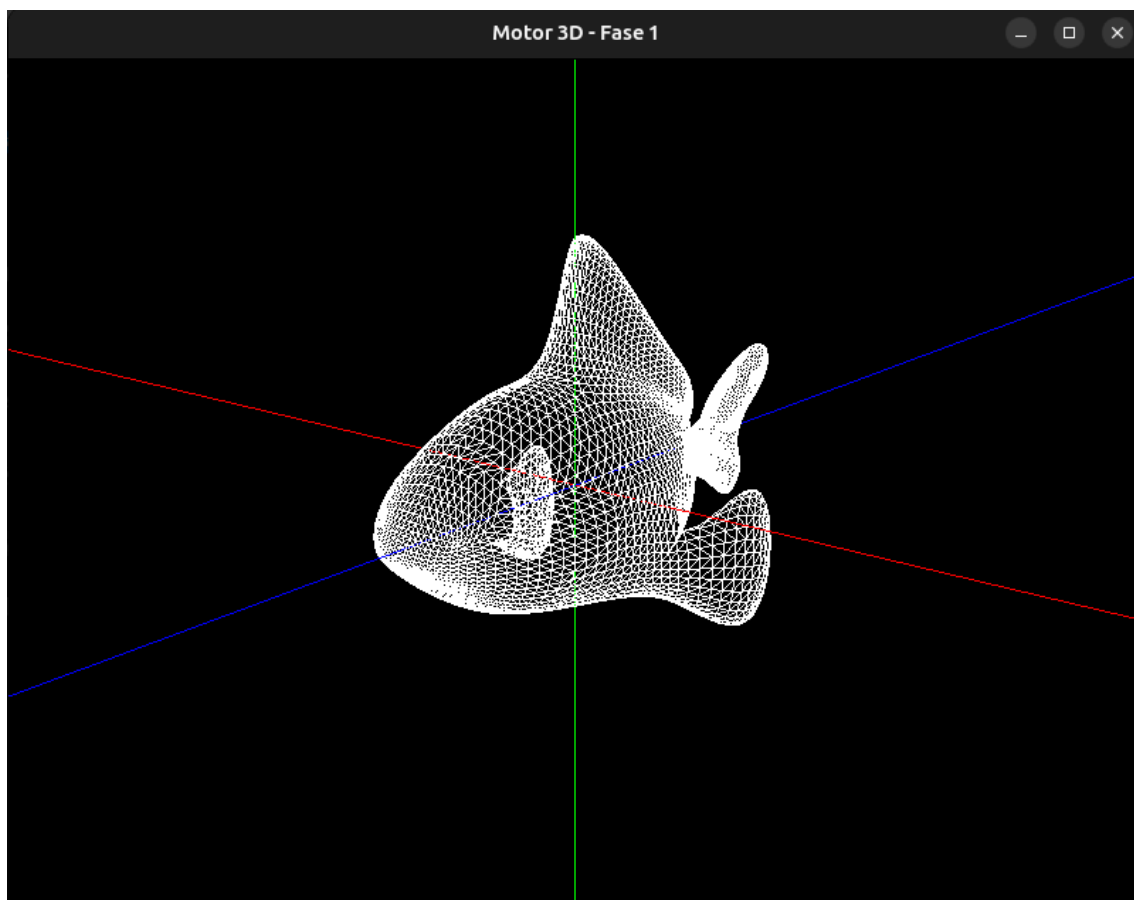


Figura 8: *Blub*.

5. Conclusão

A **Fase 1** deste projeto termina com a implementação bem-sucedida de um sistema capaz de **gerar e visualizar primitivas 3D**. A arquitetura escolhida provou ser eficaz ao separar o processamento geométrico da exibição gráfica, o que garantiu um fluxo de dados (*pipeline*) coerente entre as ferramentas. O motor consegue interpretar com rigor a matemática produzida pelo gerador e apresentá-la de forma correta através da configuração por ficheiros XML.

5.1. Desafios Encontrados

O desenvolvimento desta fase trouxe desafios técnicos significativos, especialmente pela necessidade de trabalhar sem as abstrações de motores comerciais:

- **Rigor Matemático:** A discretização de superfícies curvas, como a Esfera e o Cone, exigiu um cuidado redobrado com a trigonometria. Foi necessário garantir que as malhas de triângulos fechassem as superfícies sem falhas visíveis ou sobreposições de vértices.
- **Gestão de Memória:** O carregamento de milhares de pontos do disco para a memória RAM obrigou ao uso rigoroso de estruturas `std::vector`. O objetivo foi assegurar a estabilidade do motor e evitar fugas de memória durante a execução.
- **Pipeline de Dados:** A criação de um formato de ficheiro próprio (.3d) exigiu a definição de uma estrutura de leitura que fosse simples para o motor, mas eficiente o suficiente para não atrasar o arranque da aplicação.

6. Referências Bibliográficas

- [1] D. Shreiner, M. Woo, J. Neider, e T. Davis, *OpenGL Programming Guide*, 8.º ed. Addison Wesley, 2013.
- [2] E. W. Weisstein, «Sphere». [Em linha]. Disponível em: <https://mathworld.wolfram.com/Sphere.html>
- [3] E. W. Weisstein, «Cone». [Em linha]. Disponível em: <https://mathworld.wolfram.com/Cone.html>
- [4] L. Thomason, «tinyxml2». [Em linha]. Disponível em: <https://github.com/leethomason/tinyxml2>
- [5] Library of Congress, «Wavefront OBJ File Format». [Em linha]. Disponível em: <https://www.loc.gov/preservation/digital/formats/fdd/fdd000508.shtml>
- [6] K. Crane, «Keenan's 3D Model Repository». [Em linha]. Disponível em: <https://www.cs.cmu.edu/~kmcrane/Projects/ModelRepository/>
- [7] M. McGuire, «Computer Graphics Archive». [Em linha]. Disponível em: <https://casual-effects.com/data/>

7. Anexos

Nesta secção encontram-se elementos de suporte adicionais que, pela sua extensão ou especificidade técnica, não foram incluídos no corpo principal do relatório, mas que são relevantes para uma compreensão exaustiva do projeto.

7.1. Anexo A - Estrutura Interna do Ficheiro .3d (Exemplo)

O bloco de texto abaixo ilustra a estrutura final do ficheiro gerado pelo nosso *Generator* para um plano simples (tamanho 2, 2 divisões). A primeira linha indica o número total de vértices, seguida das coordenadas (x, y, z) de cada vértice do triângulo.

1	24	txt
2	-1 0 -1	
3	-1 0 0	
4	0 0 0	
5	0 0 0	
6	0 0 -1	
7	-1 0 -1	
8	-1 0 0	
9	-1 0 1	
10	0 0 1	
11	0 0 1	
12	0 0 0	
13	-1 0 0	
14	0 0 -1	
15	0 0 0	
16	1 0 0	
17	1 0 0	
18	1 0 -1	
19	0 0 -1	
20	0 0 0	
21	0 0 1	
22	1 0 1	
23	1 0 1	
24	1 0 0	
25	0 0 0	

7.2. Anexo B - *Script* de Conversão (.obj para .3d)

Para validar a robustez do motor com malhas complexas e externas, foi desenvolvido um *script* auxiliar em *Python*. Este utilitário permite converter modelos no formato *standard* .obj para o formato customizado .3d lido pela *Engine*.

O principal objetivo desta ferramenta é possibilitar o teste do motor com modelos tridimensionais descarregados da *internet*, que muito recorrentemente utilizam o formato .obj. O *script* lê os dados do ficheiro original e garante que todas as faces são convertidas em triângulos, o que permite que a *Engine* visualize objetos muito mais detalhados do que as primitivas básicas geradas pelo *Generator*.

```
1  import sys
2
3  def convert_obj_to_3d(obj_path, out_path):
4      vertices = []          # Stores unique vertices (1-based index
                             # in .obj)
5      output_points = []    # Stores the final sequence of triangle
                             # vertices
6
7      try:
8          with open(obj_path, 'r') as f:
9              for line in f:
10                 parts = line.strip().split()
11                 if not parts:
12                     continue
13
14                 # Parse vertex coordinates
15                 if parts[0] == 'v':
16                     vertices.append((parts[1], parts[2], parts[3]))
17
18                 # Parse face (triangle, quad, or n-gon)
19                 elif parts[0] == 'f':
20                     # Extract vertex indices (handle "v/vt/vn" or
21                     # "v//vn" formats)
22                     # Subtract 1 because .obj indices start at 1,
23                     # Python starts at 0
24                     indices = [int(p.split('/')[0]) - 1 for p in
25                               parts[1:]]
26
27                     # Triangulate polygons using a Triangle Fan
28                     # approach
29                     # If a face has 4 points (0,1,2,3), it makes 2
30                     # triangles: (0,1,2) and (0,2,3)
31                     for i in range(1, len(indices) - 1):
```

```

27         output_points.append(vertices[indices[0]])
           # Common starting point
28         output_points.append(vertices[indices[i]])
           # Current point
29         output_points.append(vertices[indices[i+1]])
           # Next point
30
31     # Write to custom .3d format
32     with open(out_path, 'w') as f:
33         f.write(f"{len(output_points)}\n") # Line 1: Total
           number of points
34         for p in output_points:
35             f.write(f"{p[0]} {p[1]} {p[2]}\n") # Triangle
           coordinates
36
37         print(f"Success! {len(output_points)} vertices written to
           '{out_path}'.")
38         print(f"Total triangles generated: {len(output_points)//3}")
39
40     except FileNotFoundError:
41         print(f"Error: File '{obj_path}' not found.")
42     except Exception as e:
43         print(f"An error occurred: {e}")
44
45 if __name__ == "__main__":
46     if len(sys.argv) != 3:
47         print("Usage: python converter.py <input.obj> <output.3d>")
48     else:
49         convert_obj_to_3d(sys.argv[1], sys.argv[2])

```